

FSBM - Faculté des Sciences Ben M'sick
Université Hassan II de Casablanca



Rapport de Projet
MLOps & DataOps

Analyse et Prédiction des Tremblements de
Terre

Conception et déploiement d'un système MLOps complet en production

Groupe : Innovation

Nom et Prénom	Rôle Principal
Hasnae Amejoud	DevOps & CI/CD
Meriem Berrima	Data Engineering
Hajar Bentaib	Machine Learning
Hamza Chentouf	API & Monitoring

Encadré par : Pr. Ait Daoud

Année Universitaire 2025 - 2026

Date : 17/09/2026

Table des matières

1 Introduction	2
2 Équipe et Répartition des Rôles	2
2.1 Rôles et Responsabilités	2
3 Méthodologie Agile	3
3.1 Organisation en Sprints	3
3.2 Rituels et Bonnes Pratiques	4
3.3 Illustration Agile avec Jira	4
4 Architecture Technique	8
4.1 Stack Technologique	8
4.2 Diagramme d'Architecture	9
5 Pipeline de Données (DataOps)	10
5.1 Ingestion et Validation (Data Quality)	10
5.2 Transformation avec dbt	11
5.3 Orchestration avec Dagster	12
6 Modélisation et Suivi des Expériences (MLOps)	13
7 CI/CD et Déploiement	16
8 Monitoring et Observabilité	18
9 Tests et Validation	19
10 Conclusion et Perspectives	20
Annexes	21

1 Introduction

La prédiction et l'analyse des tremblements de terre représentent un défi majeur tant sur le plan scientifique qu'opérationnel. Si de nombreux modèles de Machine Learning parviennent à obtenir des résultats satisfaisants dans des environnements de recherche, notamment dans des notebooks Jupyter, leur mise en production reste un obstacle complexe.

Ce projet vise à combler ce fossé en concevant et déployant un pipeline MLOps (Machine Learning Operations) et DataOps complet et robuste. L'objectif principal est de transformer un prototype de modélisation en une solution industrielle fiable, automatisée et maintenable.

Pour ce faire, le travail a été structuré autour de cinq piliers fondamentaux :

- Data Engineering : ingestion, validation et transformation fiable des données sismiques.
- MLOps : expérimentation, suivi des métriques et gestion du cycle de vie des modèles.
- API & Backend : exposition des prédictions via une API REST performante et documentée.
- DevOps & CI/CD : automatisation des tests, du build et du déploiement des conteneurs.
- Monitoring : surveillance de la santé du système et détection de la dérive des données (Data Drift).

Ce rapport détaille l'architecture technique, les choix méthodologiques, les implémentations concrètes ainsi que les perspectives d'amélioration de notre système.

2 Équipe et Répartition des Rôles

La réussite d'un projet MLOps repose sur une collaboration interdisciplinaire. Notre groupe de 4 personnes, nommé Innovation, a adopté une approche où chaque membre possède un domaine d'expertise principal, tout en assurant une revue de code croisée pour garantir la qualité globale.

2.1 Rôles et Responsabilités

Membre	Responsabilités détaillées
Hamza Chentouf	Data Engineering & DataOps : Collecte des données brutes USGS, ingestion et validation des données (Data Quality), conception du pipeline de transformation avec dbt, orchestration des tâches avec Dagster, tests de qualité des données, modélisation des données et création de l'entrepôt DuckDB.
Hajar Bentaib	Machine Learning & API : Analyse exploratoire des données (EDA) et feature engineering, entraînement et suivi des expériences avec MLflow, optimisation des modèles de régression et classification, développement de l'API de prédiction avec FastAPI et Pydantic, génération de la documentation Swagger, endpoint /health.
Hasnae Amejoud	CI/CD & Containerization : Configuration et maintenance du pipeline CI/CD avec GitHub Actions, rédaction des workflows d'intégration continue, tests automatisés (pytest), conteneurisation de l'application avec Docker et Docker Compose, construction et push des images Docker sur Docker Hub.
Meriem Berrima	Déploiement & Monitoring : Déploiement de l'infrastructure sur la plateforme Komodo, configuration des règles de routage et des ports, gestion des webhooks de déploiement continu, mise en place du monitoring et des health checks, surveillance de la santé du système et détection du Data Drift, gestion des logs et alertes.

Table 1 – Répartition détaillée des rôles au sein de l'équipe Innovation

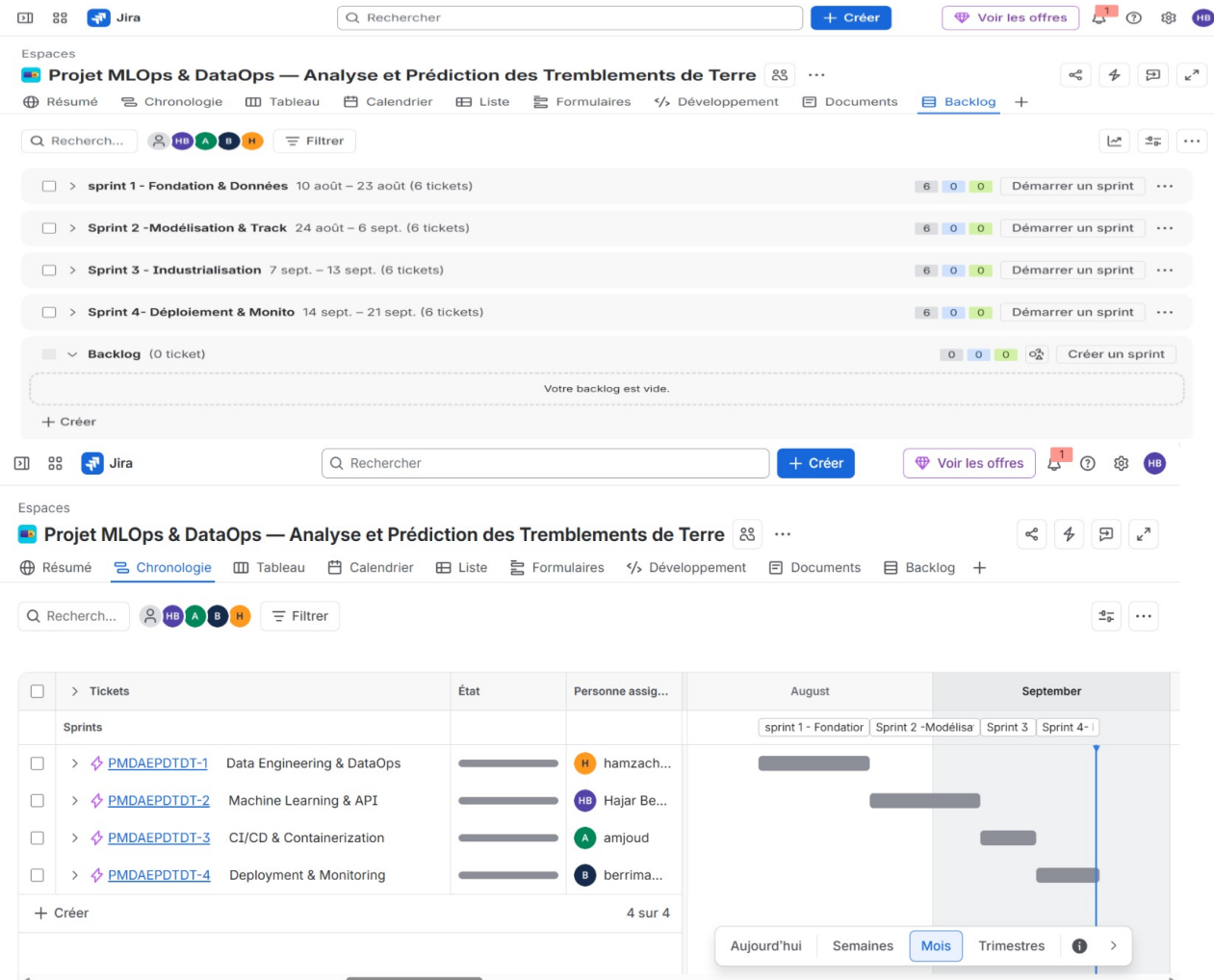
3 Méthodologie Agile

Pour gérer la complexité inhérente aux projets de données, nous avons adopté la méthodologie Scrum, adaptée à un contexte académique. Nous avons utilisé un tableau Kanban pour visualiser l'avancement des tâches à travers les colonnes To Do, In Progress, Review et Done.

3.1 Organisation en Sprints

Le projet a été découpé en 4 sprints de deux semaines, chacun livrant une valeur incrémentale :

- Sprint 1 : Fondation et Données — initialisation du repository Git, mise en place des règles de branchement (Git Flow), collecte, nettoyage et exploration initiale des données sismiques, mise en place du pipeline de transformation dbt et de l'orchestrateur Dagster.
- Sprint 2 : Modélisation et Tracking — développement et comparaison de plusieurs algorithmes (Random Forest, XGBoost, LightGBM), optimisation des hyperparamètres via Grid Search et intégration de MLflow pour enregistrer paramètres, métriques et artefacts.
- Sprint 3 : Industrialisation — développement de l'API FastAPI pour servir le modèle enregistré, écriture de tests unitaires pour valider les schémas de données et les réponses de l'API, automatisation du pipeline CI/CD.
- Sprint 4 : Déploiement et Monitoring — déploiement effectif de la stack sur la plateforme universitaire Komodo, configuration des règles de routage, des ports et des webhooks de déploiement continu, puis mise en place des premiers tableaux de bord de monitoring.



3.2 Rituels et Bonnes Pratiques

- Daily Stand-up hebdomadaire : synchronisation rapide pour lever les blocages techniques.
- Pull Requests (PR) : aucune fusion n'est effectuée sur la branche main sans revue de code par au moins un autre membre du groupe.
- Rétrospective : analyse des difficultés rencontrées et ajustement des processus.

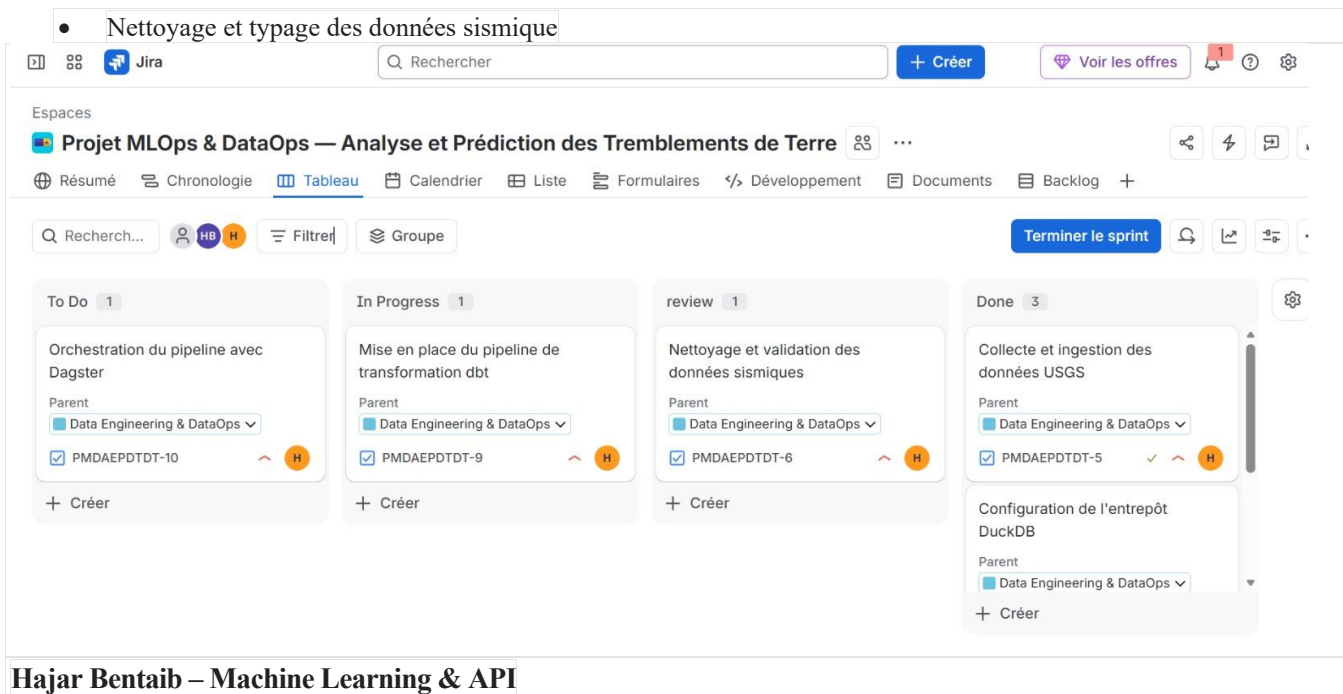
3.3 Illustration Agile avec Jira

Les tableaux ci-dessous illustrent la répartition des tâches Agile par membre. La date d'affichage est formulée comme suit : suivi Kanban des tâches au 17 septembre 2026. Ces captures sont insérées comme preuves visuelles de l'organisation du travail, tandis que l'ensemble du rapport reste éditables dans Word.

Hamza Chentouf – Data Engineering & DataOps

Tâches principales :

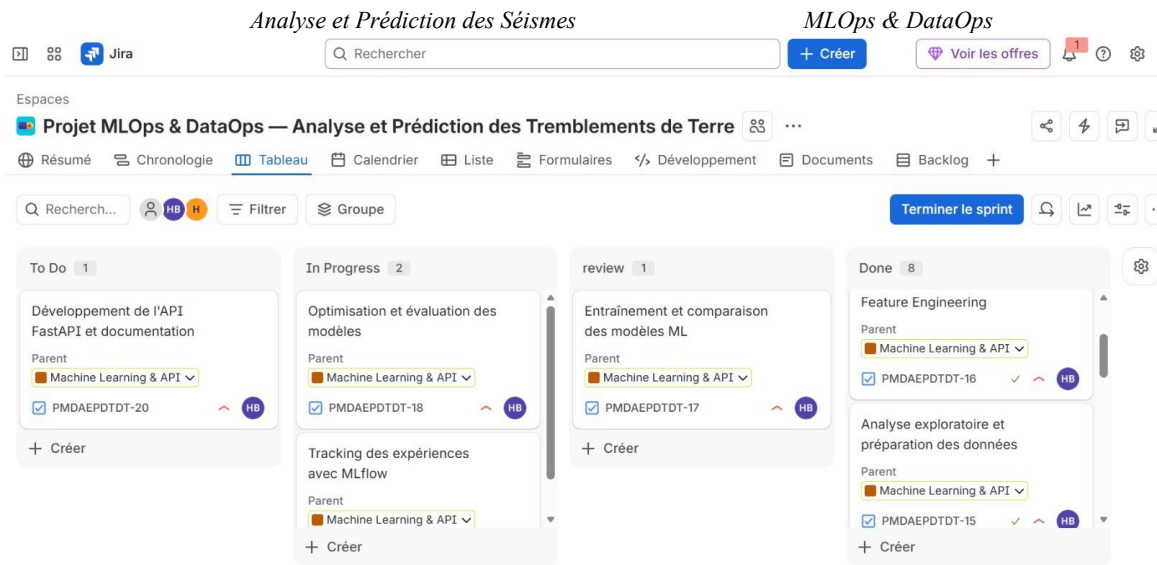
- Collecte des données brutes USGS
- Pipeline de transformation dbt (staging, intermediate, marts)
- Tests de Data Quality et validation
- Orchestration des tâches avec Dagster
- Configuration de l'entrepôt DuckDB
- Nettoyage et typage des données sismique



Hajar Bentaib – Machine Learning & API

Tâches principales :

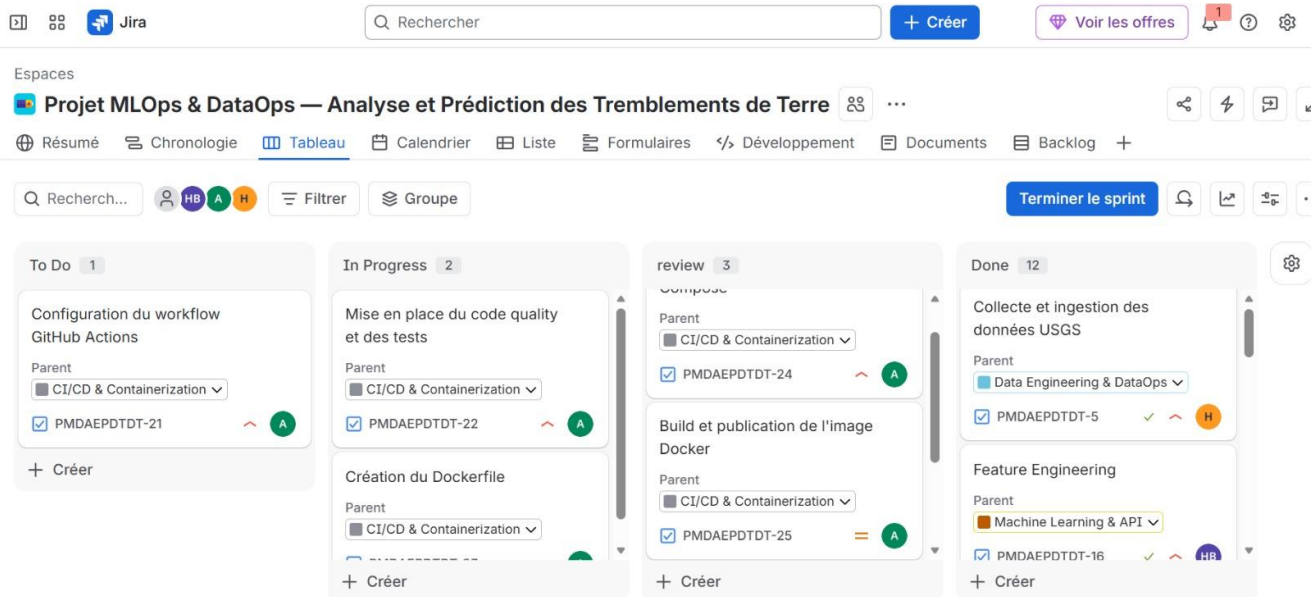
- Analyse exploratoire des données (EDA) sur les séismes
- Feature engineering et préparation des données
- Entraînement des modèles (XGBoost, Random Forest)
- Suivi des expériences avec MLflow
- Développement de l'API FastAPI
- Schémas Pydantic et validation
- Endpoint /health et documentation Swagger
- Analyse SHAP et métriques de performance



Hasnae Amejdoud – CI/CD & Containerization

Tâches principales :

- Configuration du workflow GitHub Actions
- Tests automatisés avec pytest
- Formatage du code avec Black
- Création du Dockerfile pour l'API
- Configuration du docker-compose.yml
- Build et push des images Docker sur Docker Hub
- Intégration continue et livraison continue

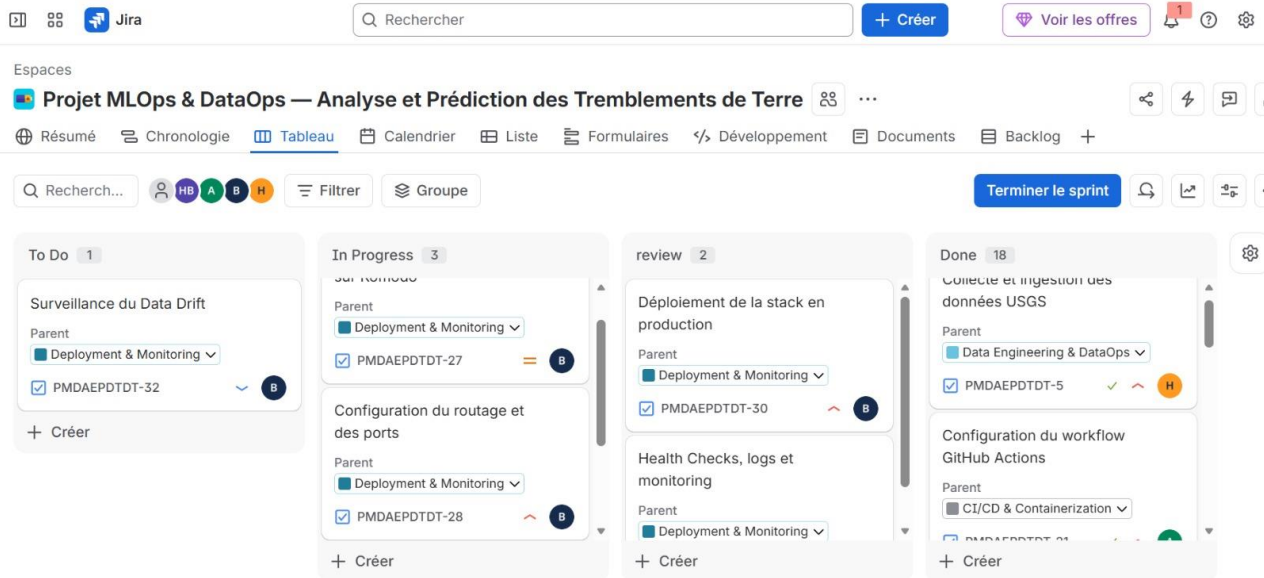


Meriem Berrima – Déploiement & Monitoring

Tâches principales :

- Déploiement de la stack sur la plateforme Komodo
- Configuration des règles de routage et des ports

- Mise en place des webhooks GitHub vers Komodo
- Health checks et surveillance du système
- Monitoring des logs et des performances
- Détection du Data Drift (perspective)
- Gestion des variables d'environnement



4 Architecture Technique

4.1 Stack Technologique

Le choix des technologies a été guidé par les principes de légèreté, de reproductibilité et de standardisation de l'industrie :

Catégorie	Technologies et justification
Langage	Python 3.11 : écosystème riche en Data Science.
DataOps	dbt pour la transformation SQL versionnée, Dagster pour l'orchestration moderne et DuckDB comme entrepôt analytique in-process.
MLOps	MLflow pour le tracking et le registre de modèles, scikit-learn, XGBoost et LightGBM pour la modélisation.
API & Backend	FastAPI pour la haute performance et la validation native via Pydantic, Uvicorn comme serveur ASGI.
DevOps	Git, GitHub Actions, Docker, Docker Compose et Komodo pour le déploiement.
Qualité	Black pour le formatage automatique, pytest pour les tests et flake8 pour le linting.
Visualisation	Streamlit pour le prototypage d'interface, Matplotlib et SHAP pour l'explicabilité.

Table 2 – Stack technologique détaillée du projet

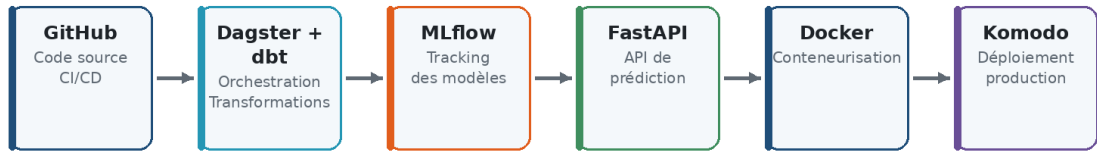
4.2 Diagramme d'Architecture

Le schéma ci-dessous illustre le flux de données et de contrôle, depuis l'ingestion jusqu'au déploiement en production. Le code source est versionné sur GitHub. À chaque modification, le pipeline CI/CD valide la qualité. Les données sont orchestrées par Dagster et transformées par dbt. Le modèle est entraîné et ses métriques sont

enregistrées dans MLflow. Une fois validé, le modèle est servi par FastAPI, conteneurisé par Docker et déployé sur Komodo.

Schéma 1 - Architecture générale du système MLOps

Flux du code, des données et du modèle jusqu'à la production



Chaîne complète : données → préparation → entraînement → API → déploiement → supervision

Chaque brique reste spécialisée tout en s'intégrant dans un flux reproductible et automatisé.

Figure 3 – Architecture générale du système MLOps

5 Pipeline de Données (DataOps)

La qualité des prédictions dépend directement de la qualité des données. Nous avons mis en place un pipeline robuste pour garantir l'intégrité des informations sismiques.

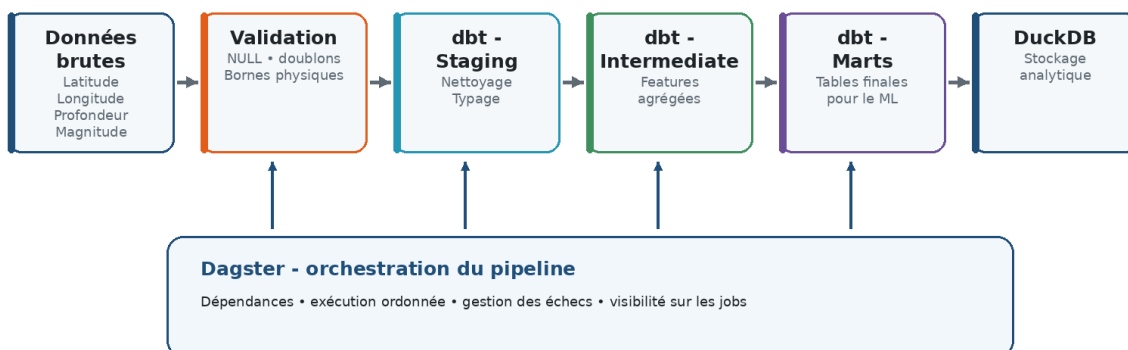
5.1 Ingestion et Validation (Data Quality)

Avant toute transformation, les données brutes subissent une série de tests de validation stricts pour s'assurer de :

- L'absence totale de valeurs nulles dans les colonnes critiques : Latitude, Longitude, Depth et Magnitude.
- Le respect des bornes géographiques et physiques réalistes : latitude entre -90 et 90, longitude entre -180 et 180, magnitude entre 0 et 10.
- L'unicité des identifiants d'événements pour éviter les doublons.

Schéma 2 - Pipeline DataOps

De la donnée sismique brute à la table finale destinée au Machine Learning



5.2 Transformation avec dbt

dbt (data build tool) est le cœur de notre ingénierie des données. Il permet de traiter les données brutes en utilisant uniquement du SQL, ce qui rend le pipeline facilement auditable et versionné. Les étapes incluent : staging pour le nettoyage et le typage, intermediate pour la création de features agrégées, et marts pour les tables finales optimisées pour l'entraînement du modèle.

```
-- models/staging/stg_earthquakes.sql
{{ config(materialized='view') }}
```

```
SELECT
    event_id,
    CAST(date AS DATE) AS event_date,
    CAST(time AS TIME) AS event_time,
    CAST(latitude AS FLOAT) AS latitude,
    CAST(longitude AS FLOAT) AS longitude,
    CAST(depth AS FLOAT) AS depth,
    CAST(magnitude AS FLOAT) AS magnitude,
    CASE
        WHEN magnitude < 5.0 THEN 'Faible'
        WHEN magnitude < 6.5 THEN 'Moyen'
        ELSE 'Fort'
    END AS magnitude_class
FROM {{ source('raw', 'earthquakes_raw') }}
WHERE depth BETWEEN 0 AND 800
    AND magnitude BETWEEN 0 AND 10
    AND event_id IS NOT NULL
```

Listing 1 – Exemple de modèle de transformation dbt (staging)

5.3 Orchestration avec Dagster

Dagster assure l'exécution ordonnée et reproductible du pipeline. Il gère les dépendances entre les tâches : l'entraînement du modèle ne peut démarrer que si la transformation dbt s'est terminée avec succès. En cas d'échec à n'importe quelle étape, Dagster interrompt le pipeline et peut déclencher des alertes.

6 Modélisation et Suivi des Expériences (MLOps)

6.1 Modèles Développés

Pour répondre à la problématique sismique, nous avons abordé le problème sous deux angles :

- Modèle de Classification : prédire la classe de dangerosité (Faible, Moyen, Fort) pour des systèmes d'alerte précoce.
- Modèle de Régression : tenter de prédire la magnitude exacte, bien que ce soit un défi en raison de la nature stochastique des séismes.

Schéma 3 - Cycle de vie MLOps

Expérimentation, sélection, déploiement et amélioration continue du modèle

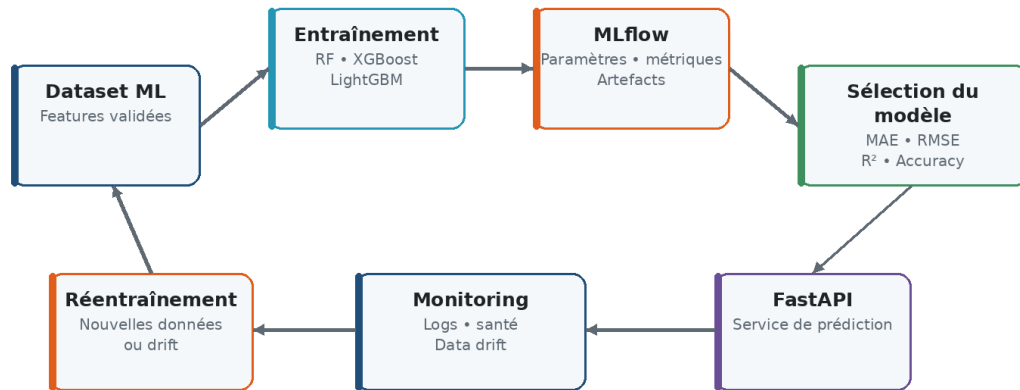


Figure 5 – Cycle de vie MLOps : expérimentation, tracking, validation et déploiement

6.2 Suivi avec MLflow

L'utilisation de MLflow est centrale dans notre démarche MLOps. Chaque exécution d'entraînement enregistre automatiquement les paramètres du modèle, les métriques de performance et les artefacts tels que le modèle sérialisé et les graphiques d'importance des features.

```

import mlflow
import mlflow.sklearn
from sklearn.metrics import mean_absolute_error, r2_score

with mlflow.start_run(run_name="xgboost_regression_v1"):
    mlflow.log_param("n_estimators", 100)
    mlflow.log_param("max_depth", 10)

    model = XGBRegressor(n_estimators=100, max_depth=10)
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    mlflow.log_metric("mae", mae)
    mlflow.log_metric("r2", r2)
    mlflow.sklearn.log_model(model, artifact_path="model")
  
```

Listing 2 – Exemple d'intégration de MLflow dans le script d'entraînement

6.3 Performances Actuelles et Analyse

Modèle	MAE	RMSE	R² / Accuracy
Régression (XGBoost)	0.305	0.418	0.034
Classification (Random Forest)	-	-	0.78 (Accuracy)

Table 3 – Performances actuelles des modèles sur le jeu de test

Analyse critique : le score R^2 de 0.034 pour la régression est faible. Cela est scientifiquement cohérent : la magnitude exacte d'un séisme dépend de facteurs géologiques profonds non capturés par nos features actuelles.

Cependant, le modèle de classification atteint une accuracy de 78 %, ce qui est pertinent pour un système d'alerte catégorielle.

6.4 Plan d'Amélioration du R²

1. Feature Engineering avancé : ajout de features polynomiales, d'interactions entre la profondeur et la localisation, et de features temporelles.
2. Algorithmes d'ensemble : mise en place d'un modèle de stacking combinant les prédictions de XGBoost, LightGBM et d'un réseau de neurones simple.
3. Sélection de features : utilisation des valeurs SHAP pour éliminer les variables bruyantes et réduire le surajustement.

7 CI/CD et Déploiement

L'automatisation est la clé pour éviter les erreurs humaines et garantir que le code en production est exactement celui qui a été testé.

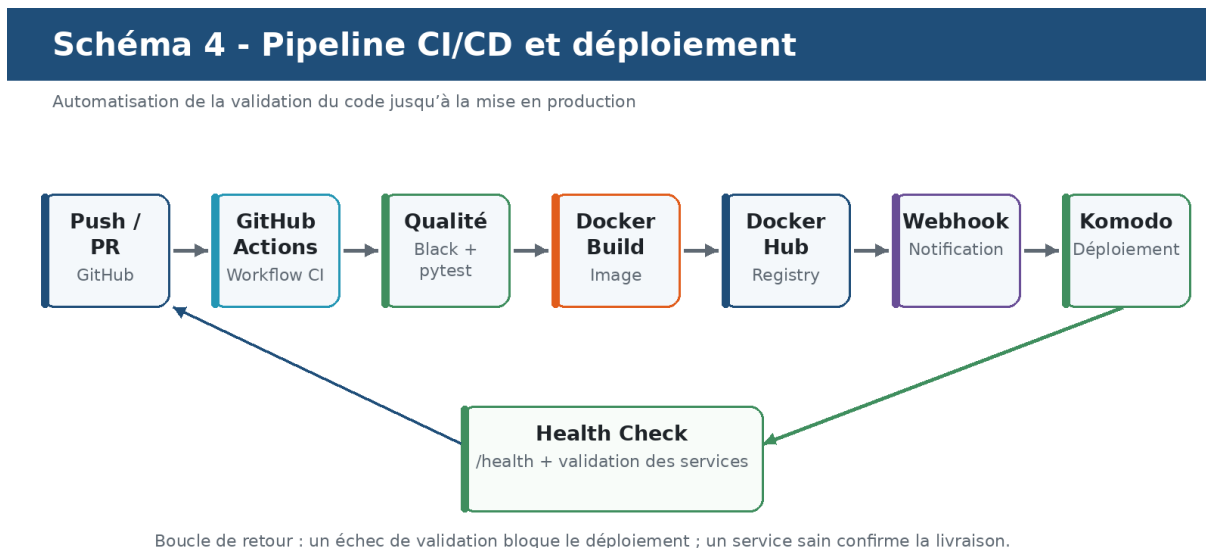


Figure 6 – Pipeline CI/CD et déploiement automatisé

7.1 Intégration Continue (CI)

À chaque push ou pull request sur la branche main, un workflow GitHub Actions se déclenche pour exécuter :

4. Checkout : récupération du code source.
5. Setup : installation de Python 3.11 et des dépendances.
6. Linting : vérification du respect des standards de code avec black --check.
7. Tests : exécution de la suite pytest pour valider la logique métier et l'API.

```

name: CI/CD Pipeline
on:
  push:
    branches: [main]

jobs:
  code-quality-and-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

```

```

- name: Set up Python
  uses: actions/setup-python@v5
  with:
    python-version: '3.11'
- name: Install dependencies
  run: pip install -r requirements.txt
- name: Check formatting with Black
  run: black --check src api
- name: Run tests with pytest
  run: pytest tests/ -v

```

Listing 3 – Extrait de la configuration CI (code-quality-and-tests)

7.2 Livraison Continue (CD)

Si et seulement si les étapes CI réussissent, le job de déploiement prend le relais :

8. Connexion sécurisée à Docker Hub en utilisant des secrets GitHub.
9. Construction de l'image Docker optimisée.
10. Push de l'image avec le tag latest vers le registry : hasnaeamejoud/earthquake-mlops:latest.

7.3 Déploiement sur Komodo

Le déploiement final s'effectue sur la plateforme universitaire Komodo via Docker Compose. Nous avons configuré un webhook dans les paramètres du Stack Komodo. Ainsi, dès que l'image est mise à jour sur Docker Hub, Komodo peut être notifié pour tirer la nouvelle image et redémarrer les services sans interruption de service.

```

version: "3.9"
services:
  mlflow:
    image: python:3.11-slim
    restart: always
    ports:
      - "5000:5000"
    command: >
      bash -c "pip install --no-cache-dir mlflow &&
      mlflow server --host 0.0.0.0 --port 5000
      --backend-store-uri /mlflow/mlruns
      --default-artifact-root /mlflow/mlruns"

  api:
    image: hasnaeamejoud/earthquake-mlops:latest
    restart: always
    ports:
      - "8000:8000"
    environment:
      - MLFLOW_TRACKING_URI=http://mlflow:5000
    depends_on:
      - mlflow

```

Listing 4 – Configuration docker-compose.yml pour la production

8 Monitoring et Observabilité

Un modèle en production n'est pas une entité statique. Nous avons mis en place des mécanismes pour assurer sa fiabilité dans le temps.

Schéma 5 - Monitoring et observabilité

Surveiller le service, les performances et l'évolution des données

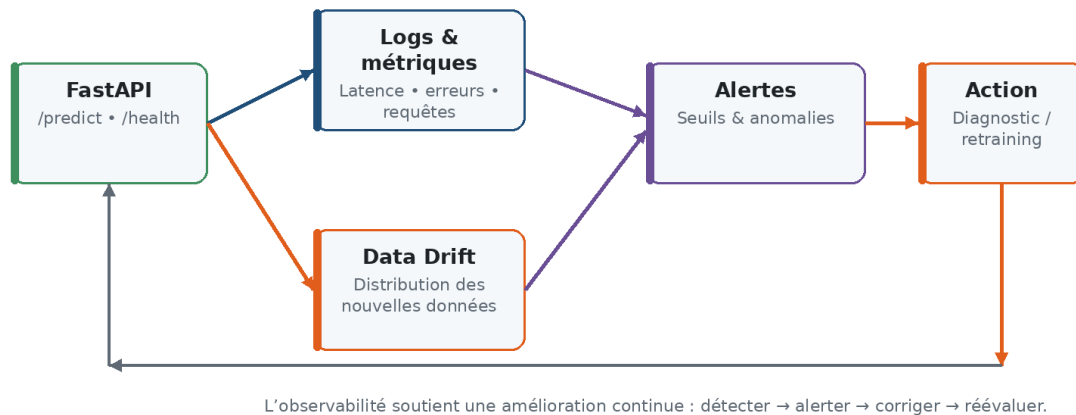


Figure 7 – Monitoring, logs, health checks et détection du data drift

8.1 Health Checks et Logs

L'API expose un endpoint /health qui vérifie non seulement que le serveur répond, mais aussi que les dépendances internes, comme la connexion au registre de modèles, sont opérationnelles. De plus, tous les accès et erreurs sont journalisés pour faciliter le débogage post-déploiement.

8.2 Data Drift (Perspective Future)

Conscients que les modèles de Machine Learning se dégradent avec le temps, nous prévoyons d'intégrer une surveillance de la distribution des données. Si la moyenne ou la variance des nouvelles données sismiques s'éloigne significativement des données d'entraînement, une alerte sera émise pour déclencher un réentraînement du modèle.

9 Tests et Validation

La robustesse du système est assurée par une pyramide de tests :

- Tests Unitaires : validation des fonctions de preprocessing et des schémas Pydantic.
- Tests d'Intégration API : utilisation de TestClient (FastAPI) pour simuler des requêtes HTTP et vérifier les codes de statut 200 OK et 422 Unprocessable Entity.
- Tests de Données : intégrés dans le pipeline dbt pour garantir qu'aucune donnée corrompue n'atteint le modèle.

10 Conclusion et Perspectives

10.1 Bilan du Projet

Ce projet a été une immersion complète dans les réalités de l'ingénierie MLOps moderne. Nous avons réussi à transformer un concept théorique en un système fonctionnel, automatisé et déployé. Les compétences acquises couvrent l'ensemble de la chaîne de valeur : de la manipulation SQL avancée avec dbt à l'orchestration avec Dagster, en passant par la conteneurisation et le CI/CD.

10.2 Défis Rencontrés

Le principal défi technique a été le déploiement sur la plateforme Komodo, notamment la gestion des permissions d'accès au terminal et la configuration des webhooks GitHub. Ces obstacles ont été surmontés grâce à une approche itérative et à l'utilisation de l'éditeur Web de Komodo pour injecter directement la configuration Docker Compose.

10.3 Perspectives d'Amélioration

11. Continuous Training (CT) : automatiser le réentraînement du modèle via Dagster dès qu'un seuil de Data Drift est franchi ou qu'un nouveau volume de données est disponible.
12. Interface Utilisateur : développer un tableau de bord interactif avec Streamlit pour permettre aux sismologues d'interroger le modèle sans connaissances techniques.
13. Monitoring Avancé : intégrer une stack Prometheus et Grafana pour visualiser en temps réel la latence de l'API et l'utilisation des ressources du serveur.

Bibliographie

- Huyen, Chip. (2022). Designing Machine Learning Systems. O'Reilly Media.
- Sculley, D. et al. (2015). Hidden Technical Debt in Machine Learning Systems. NIPS.
- Google Cloud. (2023). MLOps: Continuous delivery and automation pipelines. <https://cloud.google.com/architecture/mlops-on-gcp>
- dbt Labs. (2024). dbt Documentation. <https://docs.getdbt.com>
- Dagster. (2024). Dagster Documentation. <https://docs.dagster.io>
- MLflow. (2024). MLflow Documentation. <https://mlflow.org/docs>
- FastAPI. (2024). FastAPI Documentation. <https://fastapi.tiangolo.com>

Annexes

Annexe A : Commandes Utiles pour le Développement

```
# 1. Construire l'image Docker localement
docker build -t earthquake-mlops .

# 2. Lancer la stack complète en arrière-plan
docker compose up -d

# 3. Afficher les logs de l'API en temps réel
docker logs -f mlops-project-api-1

# 4. Exécuter les tests avec couverture de code
pytest tests/ -v --cov=src

# 5. Formater automatiquement le code
black src api tests
```

Annexe B : Liens Utiles du Projet

- Repository GitHub : <https://github.com/meriem-lgtm/mlops-project>
- Docker Hub : <https://hub.docker.com/r/hasnaeamejoud/earthquake-mlops>
- API Documentation (Swagger) : <http://komodo.s3.fsbm.ma:PORT/docs>

- MLflow UI : <http://komodo.s3.fsbm.ma:5000>